

TUSMO

Application Web de Jeu de Mots Multijoueur

DANIELIS Loïs
GOUGAUD Alexandre
Groupe L12

1. Introduction et Présentation du Jeu

1.1 Présentation du Jeu

Tusmo est un jeu de devinette de mots. Le principe est simple :

- Un mot secret est tiré aléatoirement depuis un dictionnaire français. La première lettre du mot est révélée d’emblée.
- Chaque joueur dispose de 6 tentatives pour trouver ce mot.
- Après chaque proposition, chaque lettre reçoit un code couleur : verte (lettre bien placée), jaune (lettre présente mais mal placée) ou grise (lettre absente du mot).
- La partie est divisée en plusieurs rounds (configurable). Un classement est établi à la fin en fonction du nombre d’essais et du temps de résolution.

Plusieurs joueurs partagent la même salle de jeu (Room), voient en temps réel la progression de leurs adversaires, et peuvent échanger via un chat intégré.

1.2 Fonctionnalités Principales

Fonctionnalité	Description
Authentification	Inscription et connexion par identifiant/mot de passe. Statut utilisateur (ONLINE, OFFLINE, IN_GAME) géré dynamiquement.
Gestion des Salles	Création d’une salle avec code unique à 6 caractères, rejoindre une salle existante, gestion du nombre maximum de joueurs.
Invitations	Système d’invitation entre joueurs (PENDING, ACCEPTED, REFUSED) avec notifications WebSocket.
Partie Multijoueur	Lancement de la partie par le propriétaire de la salle, déroulement synchronisé en rounds.
Chat en Direct	Messagerie textuelle intégrée dans la salle de jeu, diffusée en temps réel à tous les participants.
Classement & Scores	Calcul des points basé sur le nombre d’essais. Affichage d’un podium à la fin de chaque partie.
Historique	Consultation des parties jouées et des scores associés par salle (Pas totalement implémenté)

2. Architecture Logicielle et Structure du Code

2.1 Vue d’Ensemble : Séparation Front-end / Back-end

L’application repose sur une architecture client-serveur : le back-end et le front-end sont deux projets indépendants qui communiquent exclusivement via des interfaces bien définies.

Couche	Technologie	Rôle
Back-end	Spring Boot (Java 17)	API REST, logique métier, WebSocket
Front-end	React + TypeScript (Vite)	Interface utilisateur, appels API
Communication REST	HTTP/JSON via Fetch API	Gestion des entités (utilisateurs, salles, scores...)
Communication temps réel	STOMP over SockJS (WebSocket)	Gère les évènements de jeu
Base de données	H2 (fichier persistant)	Stockage JPA de toutes les entités

2.2 Architecture MVC côté Back-end (Spring Boot)

Le back-end applique le patron Modèle-Vue-Contrôleur (MVC), vu en cours.

Couche Controller

Les contrôleurs REST exposent les points d'entrée HTTP et délèguent immédiatement le traitement aux services. Ils ne contiennent aucune logique métier.

- UserController, GameController, RoundController, GuessController, ScoreController, RoomController, MessageController, InvitationController

Couche Service

Les services contiennent toute la logique métier.

- GameService, GuessService, NotificationService, RoomService, UserService, ScoreService, InvitationService, MessageService, RoundService

Couche Repository

Chaque entité JPA possède un repository Spring (étendant JpaRepository).

Exemples de méthodes utilisées:

- GameRepository.findByRoomIdAndStatut() : détection d'une partie active dans une salle
- GuessRepository.findByRoundIdAndUserId() : récupération des tentatives d'un joueur pour un round donné
- WordRepository.findRandomWord() : sélection aléatoire d'un mot dans le dictionnaire

Couche Entité

Les entités JPA sont annotées @Entity, utilisent Lombok (@Data, @Builder) pour éliminer le code répétitif.

2.3 Communication WebSocket

L'utilisation de WebSocket permettent de communiquer en temps réel. Le back-end configure deux canaux :

- /topic/room/{code} : canal de diffusion pour tous les joueurs d'une salle
- /topic/user/{destinataireId} : canal privé par utilisateur sur lequel le serveur pousserait un évènement INVITATION_RECEIVED dès qu'un joueur l'invite.

Les évènements WebSocket émis par le serveur sont typés via l'enum WebSocketMessage.EventType, on retrouve :

Évènement	Déclencheur	Données transmises
GAME_STARTED	Lancement de la partie par le propriétaire	gameId, roundId, première lettre, longueur du mot, nombre de rounds
GUESS_MADE	Soumission d'une tentative par un joueur	username, numéro d'essai, est correct (booléen)
ROUND_ENDED	Tous les joueurs ont fini le round	numéro du round, mot correct révélé
NEW_ROUND	Début du round suivant	roundId, première lettre, longueur, numéro de round
GAME_ENDED	Fin du dernier round	Classement complet (rang, username, points, essais)
NEW_MESSAGE	Envoi d'un message de chat	username, contenu du message

2.4 Structure du Projet Front-end (React + TypeScript)

Le front-end est structuré selon les bonnes pratiques React en séparant les responsabilités :

Dossier / Fichier	Rôle
src/pages/	Vues principales : LoginPage, LobbyPage, RoomPage, GamePage, HistoryPage

src/components/	Composants réutilisables
src/services/api.ts	Gère tous les appels REST vers le back-end
src/services/websocket.ts	Gestion de la connexion STOMP/SockJS et réception des évènements
src/context/AuthContext.tsx	Gère l'authentification (utilisateur courant)
src/hooks/useToast.tsx	Hook personnalisé pour les notifications

3. Modèle de Données

3.1 Vue d'Ensemble

Le modèle de données compte 9 entités complétées par 4 énumérations de statut. Chaque entité est justifiée par un besoin fonctionnel précis. Ces entités permettent d'implémenter la logique de notre base de données, ainsi que celle du jeu.

3.2 Description des Entités

Entité	Attributs	Justification
users	id, username (unique), email (unique), password, avatar, dateInscription, statut (UserStatus)	Représente un joueur inscrit.
words	id, mot (unique), longueur, difficulté (Difficulty)	Stocke le dictionnaire français (chargé depuis Lexique4.tsv). La difficulté permet d'envisager des modes de jeu avancés.
rooms + room_players	id, nom, code (unique 6 car.), statut (RoomStatus), maxJoueurs, owner (User), players (List<User>)	Espace de jeu partagé. La relation ManyToMany avec User gère la liste des participants.
games	id, room (Room), dateDebut, dateFin, nombreRoundsTotal, roundActuel, statut (GameStatus)	Une partie dans une salle. Découpée en rounds numérotés.
rounds	id, game (Game), word (Word), numeroRound, dateDebut, dateFin	Un round associe un mot à une partie.
guesses	id, round (Round), user (User), motPropose, dateGuess, estCorrect, resultatLettres (C/P/A)	Enregistre chaque tentative. resultatLettres (ex. « C,A,P,C,A ») stocke le résultat lettre par lettre pour reconstruction de la grille.
scores	id, user (User), game (Game), points, tempsTotal, nombreEssais	Sert à générer le classement final et l'historique. (tempsTotal n'est pas géré)
messages	id, room (Room), user (User), contenu, dateEnvoi	Persiste l'historique du chat de salle.
invitations	id, expéditeur (User), destinataire (User), room (Room), statut (InvitationStatus), dateEnvoi	Mécanisme d'invitation entre joueurs. Les statuts PENDING/ACCEPTED/REFUSED suivent l'état d'une invitation.

3.3 Énumérations de Statut

Énumération	Valeurs
UserStatus	ONLINE, OFFLINE, IN_GAME
RoomStatus	WAITING, IN_PROGRESS, FINISHED, CLOSED
GameStatus	IN_PROGRESS, FINISHED
InvitationStatus	PENDING, ACCEPTED, REFUSED
Difficulty	EASY, MEDIUM, HARD